DQ-MPCC: Dual-Quaternion MPCC for Quadrotor Racing

Bryan S. Guevara†, Luis F. Recalde†, Guanrui Li, and Tiago Nascimento

***Abstract*—We present Dual-Quaternion Model Predictive Contouring Control (DQ-MPCC) for autonomous quadrotor racing. The formulation combines dual-quaternion pose and body-frame twist dynamics with online path-progress optimization and a lag–contouring decomposition of the pose error in the desired body frame. We evaluate DQ-MPCC against a conventional MPCC baseline through Monte Carlo software-in-the-loop simulations and real-world flights on an eight-gate circuit. Both controllers run onboard at 100 Hz. Under the evaluated configuration-selection protocol, DQ-MPCC reduces the minimum lap time by 6.7% in simulation and 10.5% in real-world experiments. Without sim-to-real retuning, six of seven DQ-MPCC attempts and six of eleven baseline attempts completed all three laps. Among the completed flights, every DQ-MPCC gate crossing remained within the prescribed geometric margin, whereas the baseline exceeded the margin in four of its six completed flights. These results demonstrate favorable racing performance and sim-to-real transfer for the evaluated controller configurations.**

***Index Terms*—Aerial Systems: Mechanics and Control, Optimization and Optimal Control, Integrated Planning and Control, Motion and Path Planning.**

Supplementary Material

Code: https://github.com/LASER-Robotics/dq_mpcc

I. Introduction

MINIMUM-TIME flight through a gate sequence is the standard autonomous racing benchmark [1]–[4]. Controllers operate under strong translational and rotational coupling, while still passing through every gate [5]. Nonlinear MPC now runs onboard in real time with performance competitive with learned racing policies [2], [3], [6].

Model Predictive Contouring Control (MPCC) [7], [8] has become a standard formulation for minimum-time racing because it avoids prescribing a fixed time allocation along the reference path. MPCC optimizes the path progression online, advancing as fast as the constraints allow [9]–[11]. Existing quadrotor MPCC formulations typically express the translational contouring error in inertial coordinates and the attitude error in a separate rotational representation [9]–[11], MPCC++ [11] included, which adds a terminal safety set, learned residual dynamics, and Bayesian tuning; position and attitude remain physically coupled through the dynamics.

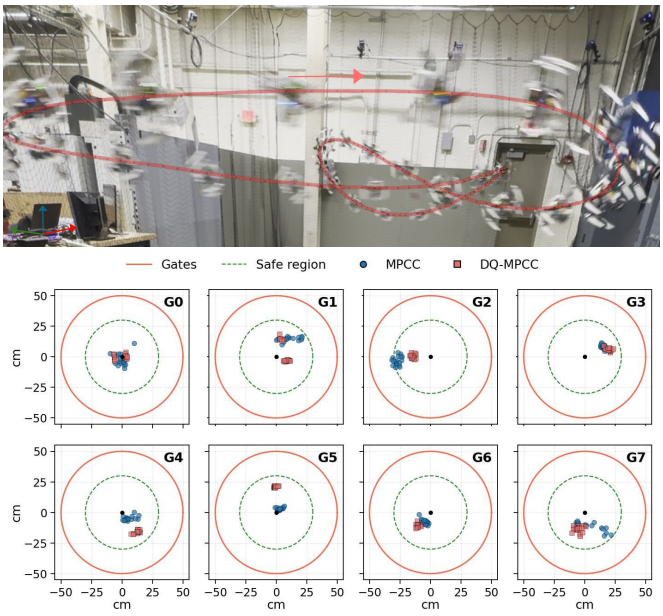


Fig. 1: Real-world experiment of the proposed **DQ-MPCC** through the 8-gate circuit (top). The bottom plots show the gate crossings of all racing laps, with red markers for the **DQ-MPCC** and blue markers for the MPCC baseline.

Under high-tilt maneuvers, reducing an inertial position error may require rapid attitude changes, so the position and attitude objectives are coupled where gate margins are tightest. Both formulations describe the same rigid-body physics; the comparison evaluates the operating points obtained for each formulation under the same tuning and deployment protocol.

Unit dual quaternions provide a unified rigid-body pose representation whose exact logarithmic map represents translational and rotational errors jointly in a tangent space isomorphic to $\mathfrak{se}(3)$, the Lie algebra of the pose group $SE(3)$ [12], [13]. Geometric controllers built on this representation enable singularity-free tracking of coupled rigid-body motion [14], [15]. A related line formulates MPC directly on matrix Lie groups [16], [17]; we bring this on-manifold perspective to contouring control. Recent work on DQ-NMPC [18] has further demonstrated the effectiveness of dual-quaternion formulations for aggressive quadrotor trajectory tracking. Despite these advances, dual-quaternion formulations have not, to our knowledge, been investigated for minimum-time quadrotor racing. Relative to the time-indexed tracking objective of DQ-NMPC [18], DQ-MPCC makes arc-length progress a decision variable and decomposes the translational pose-error

Manuscript received: June 15, 2026; Revised September 26, 2026; Accepted November 3, 2026.

This paper was recommended for publication by Editor Cosimo Della Santina upon evaluation of the Associate Editor and Reviewers' comments.

This work has been supported by the National Council for Scientific and Technological Development – CNPq under research project No. 401097/2025-0, 443960/2024-0, 304551/2023-6, and by Petrobras under the cooperation agreement project No. 0050.0132311.25.9.

†These authors contributed equally to this work.

Bryan S. Guevara and Tiago Nascimento are with the Laboratory of Systems and Robotics Engineering (LASER), Center of Informatics (CI), Federal University of Paraíba (UFPB), João Pessoa, PB 58051-900, Brazil. bryansgue@laser.ci.ufpb.br, tiagopn@ci.ufpb.br

Luis F. Recalde and Guanrui Li are with the Department of Robotics Engineering, Worcester Polytechnic Institute (WPI), Worcester, MA 01609, USA. {lfrecalde, gli7}@wpi.edu

Digital Object Identifier (DOI): see top of this page.

coordinate along the path tangent into separately weighted lag and contouring components, on which the soft lag and variable-radius tube constraints act.

We present **DQ-MPCC**, a dual-quaternion MPCC formulation in which contouring error terms are obtained by projecting the pose error onto the tangent space of the unit dual-quaternion manifold and expressing it in the desired body frame. Our method is compatible with standard flight controllers and runs onboard in real time at $100\,\mathrm{Hz}$ with an external state estimate. We evaluate **DQ-MPCC** against an MPCC baseline with the conventional inertial position-error formulation on an 8-gate circuit in both Software-in-the-Loop (SiL) simulation, in which the onboard flight computer runs the unmodified control software while the vehicle, its state estimate, and the actuator interface are simulated in MuJoCo, and real-world (RW) experiments.

As shown in Table I, the compared MPCC and dual-quaternion methods do not formulate minimum-time quadrotor racing directly on $SE(3)$. To close this gap, this paper makes the following three contributions, building on the dual-quaternion modeling of DQ-NMPC [18]:

- We introduce **DQ-MPCC**, a quadrotor racing controller that combines dual-quaternion prediction dynamics with online path-progress optimization. Lag and contouring errors are defined in desired-body-frame pose-error coordinates, with soft lag and variable-radius tube constraints.
- We demonstrate real-time onboard execution at $100\,\mathrm{Hz}$. In Monte Carlo SiL simulations, the selected **DQ-MPCC** configuration reduces the minimum lap time by 6.7% relative to the selected conventional MPCC baseline, with both satisfying the prescribed gate-crossing margin under the evaluation protocol.
- We validate **DQ-MPCC** in real-world racing on an eight-gate circuit. Without sim-to-real retuning, it reduces the minimum lap time by 10.5%. Six of seven **DQ-MPCC** attempts complete all three laps, and every crossing in the six completed flights remains within the prescribed geometric margin. The baseline completes six of eleven attempts and exceeds the margin in four of its six completed flights.

II. Dual Quaternion Modeling of the Quadrotor

A. Dual Quaternion Algebra

We summarize only what the formulation uses; see [12], [13] and DQ-NMPC [18] for complete treatments. Quaternions $\boldsymbol{q} = q_w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} \in \mathbb{H}$ have scalar part q_w, vector part $\boldsymbol{q}_v = [x,\, y,\, z]^\top$, conjugate $\boldsymbol{q}^*$, and product $\otimes$; the *unit quaternions*

$$\mathbb{S}^3 \triangleq \{\boldsymbol{q} \in \mathbb{H} : \|\boldsymbol{q}\|{=}1\}$$

represent rotations, and the *pure quaternions*

$$\mathbb{H}_p \triangleq \{\boldsymbol{q} \in \mathbb{H} : \mathrm{Re}(\boldsymbol{q}){=}0\}$$

embed three-dimensional vectors via $\boldsymbol{a} \in \mathbb{R}^3 \mapsto [0,\, \boldsymbol{a}^\top]^\top$ [12]. A dual quaternion $\hat{\boldsymbol{q}} = \boldsymbol{q}_p + \varepsilon\, \boldsymbol{q}_d \in \mathcal{H}$ pairs a primal and a dual quaternion through the dual unit ε ($\varepsilon^2{=}0$, $\varepsilon{\neq}0$), with product

$$\hat{\boldsymbol{q}}_1 \boxtimes \hat{\boldsymbol{q}}_2 = \boldsymbol{q}_{p_1} \otimes \boldsymbol{q}_{p_2} + \varepsilon\, (\boldsymbol{q}_{p_1} \otimes \boldsymbol{q}_{d_2} + \boldsymbol{q}_{d_1} \otimes \boldsymbol{q}_{p_2})$$

and conjugate $\hat{\boldsymbol{q}}^* = \boldsymbol{q}_p^* + \varepsilon\, \boldsymbol{q}_d^*$.

The subset of unit dual quaternions is

$$\mathcal{S} \triangleq \{\hat{\boldsymbol{q}} \in \mathcal{H} : \|\boldsymbol{q}_p\| = 1,\ \langle \boldsymbol{q}_p, \boldsymbol{q}_d \rangle = 0\}\,, \tag{1}$$

the double cover of $SE(3)$, used here to represent the quadrotor pose; the pure dual quaternions $\mathcal{H}_p$, with $\mathrm{Re}(\boldsymbol{q}_p){=}\mathrm{Re}(\boldsymbol{q}_d){=}0$, represent dual twists and pose errors in Section IV.

Conventions. Quaternions follow the Hamilton convention, scalar first; rotations act body-to-world; left-invariant pose errors $\hat{\boldsymbol{q}}^e = \hat{\boldsymbol{q}}^{d*} \boxtimes \hat{\boldsymbol{q}}$ are expressed in the desired body frame, the one attached to the reference pose and distinct from the vehicle body frame of the prediction model, with the real scalar kept nonnegative (jointly on real and dual parts) so that $\|\boldsymbol{\phi}\| \le \pi$. For a rotation of angle θ about $\hat{\boldsymbol{n}}$, $q_w = \cos\frac{\theta}{2}$ and $\boldsymbol{q}_v = \hat{\boldsymbol{n}} \sin\frac{\theta}{2}$ [19]; inverting gives the rotation vector $\mathrm{Log}(\boldsymbol{q}) = 2\,\boldsymbol{q}_v \arctan(\|\boldsymbol{q}_v\|, q_w)/\|\boldsymbol{q}_v\| = \theta\hat{\boldsymbol{n}}$, smooth for $\theta < \pi$ and not continuous at $\theta = \pi$, where $\pi\hat{\boldsymbol{n}}$ and $-\pi\hat{\boldsymbol{n}}$ represent the same rotation.

B. Quadrotor Model

Let $\boldsymbol{p}, \boldsymbol{v}, \boldsymbol{\omega} \in \mathbb{R}^3$ and $\boldsymbol{q}_r \in \mathbb{S}^3$ denote the inertial position, inertial velocity, angular velocity in the body frame, and attitude quaternion of a quadrotor with mass m, respectively. We adopt a body-rate command interface, where the onboard low-level rate controller tracks $\boldsymbol{\omega}_{\mathrm{cmd}}$ according to a first-order lag with time constant τ_{rc}. The quadrotor model can be written as follows:

$$\dot{\boldsymbol{p}} = \boldsymbol{v}, \tag{2}$$

$$\dot{\boldsymbol{q}}_r = \frac{1}{2}\, \boldsymbol{q}_r \otimes \boldsymbol{\omega}, \tag{3}$$

$$\dot{\boldsymbol{v}} = \frac{f}{m}\, \boldsymbol{R}(\boldsymbol{q}_r)\, \boldsymbol{e}_3 + \boldsymbol{g}, \tag{4}$$

$$\dot{\boldsymbol{\omega}} = \frac{1}{\tau_{rc}} \big(\boldsymbol{\omega}_{\mathrm{cmd}} - \boldsymbol{\omega}\big), \tag{5}$$

with f the collective thrust, $\boldsymbol{R}(\boldsymbol{q}_r) \in SO(3)$ the rotation matrix of $\boldsymbol{q}_r$, $\boldsymbol{e}_3 = [0, 0, 1]^\top$, $\boldsymbol{g} = [0, 0, -g]^\top$, and control input $\boldsymbol{u} = [f, \boldsymbol{\omega}_{\mathrm{cmd}}^\top]^\top \in \mathbb{R}^4$.

C. Dual Quaternion Quadrotor Dynamics

The position $\boldsymbol{p}$ and attitude $\boldsymbol{q}_r$ are unified into a single unit dual quaternion

$$\hat{\boldsymbol{q}} = \boldsymbol{q}_r + \varepsilon\, \boldsymbol{q}_d \ \in\ \mathcal{S}, \tag{6}$$

where $\boldsymbol{q}_d \triangleq \frac{1}{2}\, \boldsymbol{p} \otimes \boldsymbol{q}_r$, with $\boldsymbol{p}$ taken as a pure quaternion, and $\boldsymbol{q}_r$ is the unit quaternion. The position is recovered as $\boldsymbol{p} = \big(2\, \boldsymbol{q}_d \otimes \boldsymbol{q}_r^*\big)\big|_{\mathrm{vec}}$ ($|_{\mathrm{vec}}$: vector part).

With the dual-quaternion kinematics naturally in the body frame, we use the velocity in the body frame $\boldsymbol{v}_b = \boldsymbol{R}^\top(\boldsymbol{q}_r)\boldsymbol{v}$ and the dual twist

$$\hat{\boldsymbol{\xi}} = \boldsymbol{\omega} + \varepsilon\, \boldsymbol{v}_b \in \mathcal{H}_p,$$

where $\boldsymbol{\omega}$ and $\boldsymbol{v}_b$ are defined as pure quaternions. With this convention, the pose kinematics is written compactly as

$$\dot{\hat{\boldsymbol{q}}} = \frac{1}{2}\, \hat{\boldsymbol{q}} \boxtimes \hat{\boldsymbol{\xi}}. \tag{7}$$

Differentiating $\boldsymbol{v}_b$ and substituting (4)–(5) gives $\dot{\hat{\boldsymbol{\xi}}} = \hat{\boldsymbol{f}} + \hat{\boldsymbol{u}}$ with vector fields

$$\boldsymbol{f}_\omega = -\tfrac{1}{\tau_{rc}}\,\boldsymbol{\omega}, \qquad \boldsymbol{f}_v = \boldsymbol{v}_b \times \boldsymbol{\omega} - \boldsymbol{R}^\top(\boldsymbol{q}_r)\, g\, \boldsymbol{e}_3, \tag{8}$$

$$\boldsymbol{u}_\omega = \tfrac{1}{\tau_{rc}}\,\boldsymbol{\omega}_{\mathrm{cmd}}, \qquad \boldsymbol{u}_v = \frac{f}{m}\,\boldsymbol{e}_3, \tag{9}$$

giving the pure dual quaternions $\hat{\boldsymbol{f}} = \boldsymbol{f}_\omega + \varepsilon\, \boldsymbol{f}_v$ and $\hat{\boldsymbol{u}} = \boldsymbol{u}_\omega + \varepsilon\, \boldsymbol{u}_v$.

The quadrotor dynamics can be compactly written using dual quaternions as

$$\frac{d}{dt}\begin{bmatrix}\hat{\boldsymbol{q}}\\ \hat{\boldsymbol{\xi}}\end{bmatrix} = \begin{bmatrix}\frac{1}{2}\,\hat{\boldsymbol{q}} \boxtimes \hat{\boldsymbol{\xi}}\\ \hat{\boldsymbol{f}} + \hat{\boldsymbol{u}}\end{bmatrix}, \qquad \hat{\boldsymbol{q}} \in \mathcal{S},\ \hat{\boldsymbol{\xi}},\ \hat{\boldsymbol{f}},\ \hat{\boldsymbol{u}} \in \mathcal{H}_p. \tag{10}$$

The state and control are defined as follows:

$$\boldsymbol{x} = [\hat{\boldsymbol{q}},\ \hat{\boldsymbol{\xi}}] \in \mathcal{S} \times \mathcal{H}_p, \quad \boldsymbol{u} = [f,\ \boldsymbol{\omega}_{\mathrm{cmd}}^\top]^\top \in \mathbb{R}^4. \tag{11}$$

III. Reference Path Generation

MPCC uses a *spatial* reference indexed by the path arc-length s, so that the progress speed $v_s{=}\dot{s}$ becomes a decision variable [7], [8]. The reference pose at progress s is $\boldsymbol{\gamma}(s) = \big(\boldsymbol{p}^d(s),\ \boldsymbol{q}_r^d(s)\big) \in \mathbb{R}^3 \times \mathbb{S}^3$. We build the reference in three stages: gate layout, minimum-time point-mass model (PMM) planning, and differential-flatness lifting to an $SE(3)$ reference with arc-length reparametrization. Each run comprises entry from hover, a low-speed warm-up, three timed analysis laps (the only part entering the continuous metrics; failures count over the full run), and an exit.

a) Gate layout: The circuit has $N_g{=}8$ gates (center $\boldsymbol{p}_g$, normal $\boldsymbol{n}_g$) in an $11 \times 4.5 \times 3.65\,\mathrm{m}$ space, with a wall margin confining the path to the inner corridor of Fig. 4. Each gate has aperture $R_{\mathrm{gate}}{=}0.50\,\mathrm{m}$ [2] and geometric clearance $R_{\mathrm{safe}}{=}R_{\mathrm{gate}}{-}r_{\mathrm{drone}}{=}0.30\,\mathrm{m}$, with $r_{\mathrm{drone}}{=}0.20\,\mathrm{m}$ the propeller-tip radius.

b) Minimum-time lap: This stage shapes the racing path through the eight gates via an offline minimum-time Optimal Control Problem (OCP) for a point-mass model [1], [5], with state $[\boldsymbol{p}^d, \boldsymbol{v}^d]^\top$ and control acceleration $\boldsymbol{a}^d$:

$$\min_{T_\ell,\, \boldsymbol{a}^d(\cdot)} \quad T_\ell{+}\lambda_a \int_0^{T_\ell} \|\boldsymbol{a}^d\|^2\, dt \tag{12a}$$

$$\text{s.t.} \quad \dot{\boldsymbol{p}}^d = \boldsymbol{v}^d, \qquad \dot{\boldsymbol{v}}^d = \boldsymbol{a}^d, \tag{12b}$$

$$\boldsymbol{x}^d(0) = \boldsymbol{x}^d(T_\ell), \tag{12c}$$

$$\|\boldsymbol{p}^d(t_k) - \boldsymbol{p}_{g,k}\| \le \varepsilon_g, \tag{12d}$$

$$\boldsymbol{v}^d(t_k){\cdot}\boldsymbol{n}_{g,k} \ge v_{\min}, \tag{12e}$$

$$\|\boldsymbol{a}^d\|_2 \le 5g, \qquad a_z^d \ge -0.9g, \tag{12f}$$

for $k{=}1, \ldots, N_g$, where $\boldsymbol{x}^d{=}[\boldsymbol{p}^d, \boldsymbol{v}^d]^\top$ is the planner state, T_ℓ is the lap time, and t_k is the time at which gate k (center $\boldsymbol{p}_{g,k}$, normal $\boldsymbol{n}_{g,k}$) is crossed. The objective (12a) minimizes the lap time with a small control-effort regularization for smoothness ($\lambda_a{=}10^{-6}$, SI). The periodicity constraint (12c) yields a single repeatable lap, and (12d)–(12e) keep the path within $\varepsilon_g{=}5\,\mathrm{cm}$ of each gate center and require a forward crossing. In (12f) the bound $\|\boldsymbol{a}^d\|_2 \le 5g$ limits the planned acceleration, while the thrust-positive bound $a_z^d \ge -0.9g$ keeps the required thrust pointing upward, so the attitude reference stays differentiable once the path is lifted to a full pose by differential flatness.

The $5g$ envelope exceeds the platform's maximum thrust acceleration, so the point-mass lap cannot be tracked exactly by the real vehicle. This is intentional: MPCC does not require a dynamically feasible trajectory, only a continuously differentiable spatial path [9]. The planner therefore defines the geometric path, the speed along it being decided online under the true actuation limits.

A variable tube $d_{\mathrm{tube}}(s)$ is then constructed around the path: it narrows at each gate and widens between gates, bounding the admissible contouring error along the path.

c) Arc-length reparametrization and flatness lifting: The reference is resampled to arc-length $s(t){=}\int_0^t \|\boldsymbol{v}^d\|d\tau$ and Gaussian-smoothed ($\sigma_a{=}25\,\mathrm{ms}$) into a C^2, bounded-jerk profile $\tilde{\boldsymbol{a}}^d(s)$. Differential flatness [20] then defines the thrust axis $\boldsymbol{b}_3^d$ and, with a fixed-yaw convention $\psi^d{=}0$ standard in racing [1], [5], [9], the remaining orthonormal columns of the rotation $\boldsymbol{R}^d(s){=}[\boldsymbol{b}_1^d\, \boldsymbol{b}_2^d\, \boldsymbol{b}_3^d] \in SO(3)$:

$$\boldsymbol{b}_3^d = \frac{\tilde{\boldsymbol{a}}^d + g\,\boldsymbol{e}_3}{\|\tilde{\boldsymbol{a}}^d + g\,\boldsymbol{e}_3\|}, \quad \boldsymbol{b}_2^d = \frac{\boldsymbol{b}_3^d \times \boldsymbol{e}_1}{\|\boldsymbol{b}_3^d \times \boldsymbol{e}_1\|}, \quad \boldsymbol{b}_1^d = \boldsymbol{b}_2^d \times \boldsymbol{b}_3^d, \tag{13}$$

with $\boldsymbol{e}_1$ the world x-axis.

$\boldsymbol{q}_r^d(s)$ is extracted from $\boldsymbol{R}^d$ with sign chosen for continuity in s; the feedforward rate in the body frame is linear in the progress speed,

$$\boldsymbol{\omega}^d(s, v_s) = \boldsymbol{\omega}_0(s)\, v_s, \quad \boldsymbol{\omega}_0(s) \triangleq \left(\boldsymbol{R}^{d^\top} \tfrac{d\boldsymbol{R}^d}{ds}\right)^\vee \in \mathbb{R}^3, \tag{14}$$

where $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$ returns the vector of a skew-symmetric matrix [21]. A final Gaussian low-pass ($\sigma_q{=}85\,\mathrm{ms}$) on $\boldsymbol{q}_r^d(s)$ keeps $\boldsymbol{\omega}^d$ within the $20\,\mathrm{rad/s}$ envelope. References are C^2 cubic B-splines evaluated symbolically inside the OCP. Since $\boldsymbol{R}^d(s)$ is lifted from the planner's acceleration profile it is not recomputed as v_s departs from the planned timing, only (14) scaling with v_s; an attitude consistent with the timing would depend on (s, v_s, a_s). We retain the offline profile as a smooth C^2 heuristic soft prior, weighted through the soft attitude term and never as a constraint.

IV. DQ-MPCC

The architecture of **DQ-MPCC** and its main components are shown in Fig. 2. The offline pipeline provides the OCP with the B-splines $\boldsymbol{\gamma}(s)$ and $\boldsymbol{\omega}_0(s)$.

A. System Dynamics

We augment the pose state with the arc-length progress pair (s, v_s), letting the controller adjust speed along the path, and with the thrust f, whose rate Δf is the smooth control,

$$\dot{s} = v_s, \qquad \dot{v}_s = a_s, \qquad \dot{f} = \Delta f, \tag{15}$$

with a_s the progress acceleration. Stacked onto the dual-quaternion state (11):

$$\begin{aligned} \boldsymbol{x}_{\mathrm{a}} &= \big[\hat{\boldsymbol{q}},\ \hat{\boldsymbol{\xi}},\ s,\ v_s,\ f\big] \in \mathcal{S} \times \mathcal{H}_p \times \mathbb{R}^3,\\ \boldsymbol{u}_{\mathrm{a}} &= \big[\,\Delta f,\ \boldsymbol{\omega}_{\mathrm{cmd}}^\top,\ a_s\big]^\top \in \mathbb{R}^5. \end{aligned} \tag{16}$$

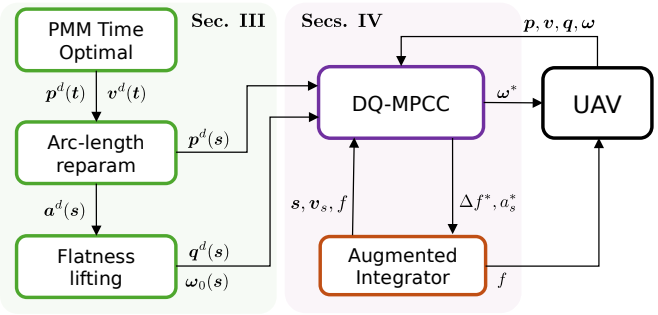


Fig. 2: Control block diagram of the proposed **DQ-MPCC**, with shaded regions illustrating the subsections where each component is presented.

B. Optimal Control Problem

Over a non-uniform grid of N shooting intervals spanning T_p, **DQ-MPCC** solves at every step:

$$\min_{\boldsymbol{x}_{\mathrm{a},0:N},\,\boldsymbol{u}_{\mathrm{a},0:N-1}} \quad \sum_{k=0}^{N-1} \ell(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k}) + \ell_e(\boldsymbol{x}_{\mathrm{a},N}) \tag{17a}$$

$$\text{s.t.} \quad \boldsymbol{x}_{\mathrm{a},k+1} = \boldsymbol{F}(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k}), \tag{17b}$$

$$\boldsymbol{x}_{\mathrm{a},0} = \boldsymbol{x}_{\mathrm{a,init}}, \tag{17c}$$

$$\boldsymbol{g}(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k}) \le \boldsymbol{0}. \tag{17d}$$

Here ℓ and ℓ_e are the stage and terminal costs, $\boldsymbol{F}(\cdot)$ the ERK4-integrated augmented dynamics, and $\boldsymbol{g}(\cdot)$ the input and state constraints. Following [22], the non-uniform grid (Table II) concentrates resolution near the current state.

C. Stage Cost

The stage cost combines the tangent-space pose error ($\boldsymbol{\phi}^e$, $\boldsymbol{\rho}_{\mathrm{lag}}$, $\boldsymbol{\rho}_{\mathrm{cont}}$), control terms, and a progress reward. With the body-rate error $\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega}^d(s, v_s) - \boldsymbol{\omega}_{\mathrm{cmd}}$ relative to the feedforward, the stage cost is defined as:

$$\begin{aligned}\ell(\boldsymbol{x}_{\mathrm{a}}, \boldsymbol{u}_{\mathrm{a}}) = {\boldsymbol{\phi}^e}^{\top} \boldsymbol{Q}_{\phi}\, \boldsymbol{\phi}^e + \boldsymbol{\rho}_{\mathrm{lag}}^{\top} \boldsymbol{Q}_{\ell}\, \boldsymbol{\rho}_{\mathrm{lag}} + \boldsymbol{\rho}_{\mathrm{cont}}^{\top} \boldsymbol{Q}_{c}\, \boldsymbol{\rho}_{\mathrm{cont}} \\ + W_{\Delta f}\, \Delta f^{\,2} + \tilde{\boldsymbol{\omega}}^{\top} \boldsymbol{W}_{\omega}\, \tilde{\boldsymbol{\omega}} - W_s\, v_s,\end{aligned} \tag{18}$$

where $\boldsymbol{Q}_{\phi}, \boldsymbol{Q}_{\ell}, \boldsymbol{Q}_{c} \in \mathbb{R}^{3\times3}$ are diagonal pose-error weights, $W_{\Delta f} > 0$ penalizes the thrust rate Δf for smoothness (cf. (15)), $\boldsymbol{W}_{\omega} \in \mathbb{R}^{3\times3}$ penalizes deviations from the angular-rate feedforward $\boldsymbol{\omega}^d(s, v_s)$ (14), and $W_s > 0$ weights progress along the path. The terminal cost ℓ_e retains only the pose-error terms of (18).

D. Dual-Quaternion Pose Error

Let $\hat{\boldsymbol{q}}^d(s)$ be the unit dual quaternion encoding the reference pose $\boldsymbol{\gamma}(s)$ via (6). The left-invariant pose error

$$\hat{\boldsymbol{q}}^e = \hat{\boldsymbol{q}}^{d\,*}(s) \boxtimes \hat{\boldsymbol{q}} \;\in\; \mathcal{S} \tag{19}$$

expresses the rotational and translational deviation jointly in the body frame of the desired pose [23]; its real part is the rotation error $\boldsymbol{q}_r^e \triangleq \boldsymbol{q}_r^{d*} \otimes \boldsymbol{q}_r$; for $\hat{\boldsymbol{q}}, \hat{\boldsymbol{q}}^d \in \mathcal{S}$ it remains unit by group closure, so the logarithmic map below is well defined; the predicted nodes leave $\mathcal{S}$ by the residuals of Section VI.

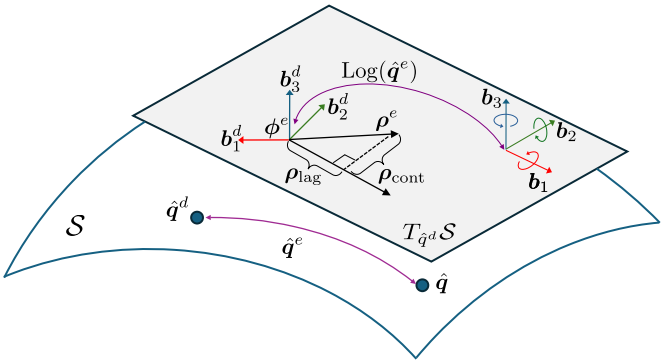


Fig. 3: Dual-quaternion pose error: $\mathrm{Log}(\hat{\boldsymbol{q}}^e)$ lifts the error between $\hat{\boldsymbol{q}}$ and $\hat{\boldsymbol{q}}^d$ on $\mathcal{S}$ onto the tangent space at the desired pose, giving $\boldsymbol{\phi}^e$ and $\boldsymbol{\rho}^e$ in the desired body frame $\boldsymbol{b}_i^d$, the vehicle frame being $\boldsymbol{b}_i$; the translational coordinate is then decomposed along the path tangent into $\boldsymbol{\rho}_{\mathrm{lag}}$ and $\boldsymbol{\rho}_{\mathrm{cont}}$.

The logarithmic map projects $\hat{\boldsymbol{q}}^e$ onto $\mathcal{H}_p$, the Lie algebra of the unit dual quaternions; its six free parameters are identified with $\mathfrak{se}(3)$ [13] and used as the error coordinates throughout:

$$\mathrm{Log}(\hat{\boldsymbol{q}}^e) = \begin{pmatrix} \boldsymbol{\phi}^e \\ \boldsymbol{\rho}^e \end{pmatrix} \in \mathbb{R}^6, \tag{20}$$

where $\boldsymbol{\phi}^e \triangleq \mathrm{Log}(\boldsymbol{q}_r^e) = \theta\hat{\boldsymbol{n}}$ is the rotation vector of the rotation error; DQ-NMPC [18] penalizes half of it, $\frac{1}{2}\boldsymbol{\phi}$, and takes the position error itself as translational part, without the inverse left Jacobian of (21). The translational coordinate $\boldsymbol{\rho}^e$ follows from the position error in the desired body frame, $\boldsymbol{t}^e \triangleq \left(2\,\boldsymbol{q}_d^e \otimes \boldsymbol{q}_r^{e\,*}\right)\big|_{\mathrm{vec}} = \boldsymbol{R}^{d\top}(\boldsymbol{p} - \boldsymbol{p}^d)$ [13], through the inverse left Jacobian of $SO(3)$ [12], [24]:

$$\boldsymbol{\rho}^e \triangleq \boldsymbol{J}_l^{-1}(\boldsymbol{\phi}^e)\, \boldsymbol{t}^e, \tag{21}$$

whose closed form, with $\boldsymbol{\phi}^e = \theta\hat{\boldsymbol{n}}$ the rotation vector and $\alpha = \frac{\theta}{2}\cot\frac{\theta}{2}$, is [19]

$$\boldsymbol{J}_l^{-1}(\theta\hat{\boldsymbol{n}}) = \alpha\, \boldsymbol{I}_3 - \tfrac{\theta}{2}\,[\hat{\boldsymbol{n}}]_{\times} + (1-\alpha)\,\hat{\boldsymbol{n}}\hat{\boldsymbol{n}}^{\top}, \tag{22}$$

which the deployed controller approximates as $\boldsymbol{J}_l^{-1} \approx \boldsymbol{I}$, so that $\boldsymbol{\rho}^e = \boldsymbol{R}^{d\top}(\boldsymbol{p} - \boldsymbol{p}^d)$ as deployed: an identity approximation of the joint $\mathfrak{se}(3)$ pose-error coordinate about zero rotation error, not the exact logarithm. Adopted as a reduced pose-error coordinate, it simplifies the translational mapping; Section VI quantifies its non-negligible racing-speed error and evaluates the exact coordinate at fixed gains.

E. Lag and Contouring Decomposition in $\mathfrak{se}(3)$

Projecting the error $\boldsymbol{\rho}^e$ in the desired body frame onto the path tangent $\boldsymbol{t}_b^d = \boldsymbol{R}^{d\top}(s)\, \boldsymbol{t}^d$, with $\boldsymbol{t}^d$ the unit tangent of the reference path, decomposes it into lag (along-path) and contouring (across-path) components (Fig. 3):

$$\boldsymbol{\rho}_{\mathrm{lag}} = \left(\boldsymbol{t}_b^d\, \boldsymbol{t}_b^{d\top}\right) \boldsymbol{\rho}^e, \qquad \boldsymbol{\rho}_{\mathrm{cont}} = \left(\boldsymbol{I}_3 - \boldsymbol{t}_b^d\, \boldsymbol{t}_b^{d\top}\right) \boldsymbol{\rho}^e. \tag{23}$$

With the rotation error $\boldsymbol{\phi}^e$, these form the pose tracking error in the cost (18), expressed throughout in the desired body frame.

TABLE I: MPCC vs DQ control formulations. Quat: unit quaternion; DQ: unit dual quaternion; PF: path following; TK: tracking. †Non-uniform grid (Section IV-B). $\mathfrak{se}(3)$ is the formulation of (21), deployed with $\boldsymbol{J}_l^{-1}{\approx}\boldsymbol{I}$.

Feature	[7]	[10]	[9]	[11]	[18]	**Ours**
Control type	PF	PF	PF	PF	TK	PF
Dynamics model	Bicycle	Point mass	Full 6-DoF	Full 6-DoF	Full 6-DoF	Full 6-DoF
Pose error space	$\mathbb{R}^2$	$\mathbb{R}^3$	$\mathbb{R}^3{\times}\mathfrak{so}(3)$	$\mathbb{R}^3{\times}\mathfrak{so}(3)$	$\mathfrak{se}(3)$	$\mathfrak{se}(3)$
Rotation repr.	—	—	Quat	Quat	DQ	DQ
Unified coords., body frame	—	—	✗	✗	✓	✓
Lag and cont. in $\mathfrak{se}(3)$	✗	✗	✗	✗	—	✓
Progress dynamics	$\dot{s}{=}v_s$	$\ddot{s}{=}a_s$	$\ddot{s}{=}a_s$	$\ddot{s}{=}a_s$	—	$\ddot{s}{=}a_s$
Safety constraints	Track bounds	Corridor	—	Gate avoid.	—	Soft lag, tube
Horizon N / T_p	40 / 0.8 s	20 / 1.0 s	20 / 0.8 s	20 / 0.8 s	—/ 1.5 s	31 / 1.48 s†
Solved onboard	—	✓	✗	✗	✓	✓

F. Constraints

The constraint function $\boldsymbol{g}(\boldsymbol{x}_\mathrm{a}, \boldsymbol{u}_\mathrm{a}) \leq \boldsymbol{0}$ in (17d) collects the actuator, progress, and corridor limits

$$f_{\min} \leq f \leq f_{\max}, \tag{24a}$$
$$|\Delta f| \leq \Delta f_{\max}, \tag{24b}$$
$$|\omega_{\mathrm{cmd},i}| \leq \omega_{\max}, \quad i = 1,2,3, \tag{24c}$$
$$a_{s,\min} \leq a_s \leq a_{s,\max}, \tag{24d}$$
$$\|\boldsymbol{\rho}_\mathrm{lag}\| \leq D_\mathrm{lag} + \sigma_\ell, \quad \sigma_\ell \geq 0, \tag{24e}$$
$$\|\boldsymbol{\rho}_\mathrm{cont}\| \leq d_\mathrm{tube} + \sigma_c, \quad \sigma_c \geq 0, \tag{24f}$$
$$0 \leq s \leq s_{\max} + s_m, \tag{24g}$$
$$0 \leq v_s \leq v_{s,\max}. \tag{24h}$$

Constraints (24e)–(24f) bound the lag and contouring errors softly (D_lag, tube radius $d_\mathrm{tube}(s)$; cost-penalized slacks); s_m is a terminal progress margin. The norm constraint in (1) is maintained by a Baumgarte-type correction $K_q(1-\|\boldsymbol{q}_r\|)\,\boldsymbol{q}_r$, $K_q{=}10\,\mathrm{s}^{-1}$, added to $\dot{\boldsymbol{q}}_r$ inside the integrator [25]; the orthogonality constraint is re-established at every control iteration, where $\hat{\boldsymbol{q}}$ is rebuilt from the estimated pose via (6).

Physical states come from the state estimator; virtual states (s, v_s, f) are Euler-integrated locally at the loop period Δt from $\boldsymbol{u}^*_{\mathrm{a},0}$,

$$\begin{aligned} s_{k+1} &= s_k + v_{s,k}\,\Delta t, \\ v_{s,k+1} &= v_{s,k} + a^*_s\,\Delta t, \\ f_{k+1} &= f_k + \Delta f^*\,\Delta t, \end{aligned} \tag{25}$$

with drift absorbed by re-initialization (17c) each iteration.

V. Experimental Setup

Numerical implementation. The OCP is solved with acados [26] via Sequential Quadratic Programming Real-Time Iterations (SQP-RTI) [27]; simulations use MuJoCo with the parameters of Table II. Inequality constraints are slack-relaxed to keep the QP feasible. All SiL evaluations run on the same Jetson Orin NX used in the real-world experiments.

Baseline. The baseline matches **DQ-MPCC** in solver, horizon, constraints, reference, actuation, and cost structure, but differs in state and pose-error coordinates: it propagates the inertial body-rate model (2)–(5), augmented with the same arc-length and thrust states (15), giving the state $\boldsymbol{x}_\mathrm{e}{=}(\boldsymbol{p}, \boldsymbol{v}, \boldsymbol{q}_r, \boldsymbol{\omega}, s, v_s, f)$ in place of the dual-quaternion state in the body frame (16), and it uses the standard $\mathbb{R}^3{\times}\mathfrak{so}(3)$ error decomposition in place of the $\mathfrak{se}(3)$ one. The inertial position error $\boldsymbol{e}_p = \boldsymbol{p}^d(s) - \boldsymbol{p}$ decouples into along-path and across-path components,

$$\boldsymbol{e}_\ell = (\boldsymbol{t}^{d\top}\boldsymbol{e}_p)\,\boldsymbol{t}^d, \qquad \boldsymbol{e}_c = (\boldsymbol{I}_3 - \boldsymbol{t}^d\,\boldsymbol{t}^{d\top})\,\boldsymbol{e}_p,$$

with attitude error $\boldsymbol{\eta} = \mathrm{Log}(\boldsymbol{q}_r^* \otimes \boldsymbol{q}_r^d) \in \mathfrak{so}(3)$ in the body frame; the baseline cost is (18) with $\boldsymbol{\eta}$, $\boldsymbol{e}_\ell$, and $\boldsymbol{e}_c$ in place of $\boldsymbol{\phi}^e$, $\boldsymbol{\rho}_\mathrm{lag}$, and $\boldsymbol{\rho}_\mathrm{cont}$, with the control and progress terms unchanged.

For the deployed approximation, this cost has the same term structure as the **DQ-MPCC** one: $\boldsymbol{R}^d$ being orthogonal, $\boldsymbol{\rho}_\mathrm{lag}{=}{-}\boldsymbol{R}^{d\top}\boldsymbol{e}_\ell$, $\boldsymbol{\rho}_\mathrm{cont}{=}{-}\boldsymbol{R}^{d\top}\boldsymbol{e}_c$, so the translational errors share the same norm. The two error quaternions are conjugates, so the attitude errors are the same rotation vector up to sign, $\boldsymbol{\phi}^e = -\boldsymbol{\eta}$, and the attitude penalties coincide under matched weights $\boldsymbol{Q}_\phi = \boldsymbol{Q}_q$. With the isotropic pose-error weights of Section V-A the cost shape therefore does not differ between the controllers. Equality of the instantaneous error norms does not make the two nonlinear programs equivalent: the errors are functions of different state coordinates, so the second derivatives of the two stage costs differ, and the prediction models propagate those coordinates differently over the horizon. Performance differences therefore cannot reflect missing physical coupling; they can arise from the coordinates in which the problem is stated and from the independently tuned gains, which remain a confound (Section VI). Comparing directly to published racing MPCCs [9]–[11] would confound solver, tuning, actuation, and platform with the $SE(3)$ effect, the two with the full model being solved off board; Table I positions both formulations instead.

Metrics. $T_\mathrm{min}/T_\mathrm{avg}$: minimum/average lap time over the N_ℓ analysis laps; δ: in-plane gate-crossing offset from the gate center, with maximum δ_max and average δ_avg over the crossings of a run. *Crossing-margin compliance* is $\delta{\leq}R_\mathrm{safe}$ at every crossing; this event metric does not guarantee collision-free traversal elsewhere. e_c, e_ℓ: contour and lag RMSE from the same inertial path projection (independent of the representation); L_p: per-lap path length; $|\boldsymbol{v}|_\mathrm{max}$, $|\boldsymbol{a}|_{p99}$, $|\boldsymbol{\omega}|_\mathrm{rms}$: maximum speed, 99th-percentile acceleration, RMS body rate over the racing laps (odometry and IMU).

TABLE II: System and OCP Parameters.

Platform		Solver / OCP	
m	1.08 kg	solver	acados SQP-RTI
τ_{rc}	30 ms	integrator	ERK4
f	$[0, 49.05]$ N	horizon N	31
$\lvert\Delta f\rvert$	$\leq$ 500 N/s	T_p	1.48 s
$\lvert\omega_{\mathrm{cmd},i}\rvert$	$\leq$ 20 rad/s	loop rate	100 Hz
$\lvert a_s\rvert$	$\leq 5g$	regularization	PROJECT+LM (10^{-2})
D_lag	0.5 m	grid [ms]	10×20+10×40+11×80
s_max	≈118 m	QP solver	PARTIAL_CONDENSING_HPIPM
$v_{s,\mathrm{max}}$	≈14.65 m/s	Hessian approx.	Gauss Newton

A. Multi-Objective Tuning

Both controllers are tuned offline with Optuna [28]: a broad multi-objective NSGA-III [29] exploration, a TPE refinement, and a final evaluation under matched progress-speed bounds $v_{s,\mathrm{max}}$, so that performance differences are not driven by different speed limits. Each stage uses 100 trials per controller and $n_\mathrm{seeds} = 3$ MuJoCo seeds.

TABLE III: SiL racing metrics, n=100 Monte Carlo seeds; median $\pm$ std over completed runs (contacts over all attempts of the solid-gate campaigns); **bold** = better of the two deployed controllers; † = same-gains ablation with the exact coordinate (21) (Section VI); solve times: overhead vs. **DQ-MPCC** measured side by side

Metric	MPCC	**DQ-MPCC**	exact $\boldsymbol{J}_l^{-1\,\dagger}$
$T_{\min}$ [s]	3.457 ± 0.010	$\mathbf{3.226 \pm 0.007}$	3.141 ± 0.012
T_{avg} [s]	3.478 ± 0.006	$\mathbf{3.252 \pm 0.004}$	3.164 ± 0.011
$\delta_{\max}$ [cm]	$\mathbf{17.88 \pm 0.62}$	24.53 ± 0.73	24.55 ± 0.80
δ_{avg} [cm]	$\mathbf{12.01 \pm 0.20}$	15.85 ± 0.20	15.44 ± 0.44
e_c RMSE [cm]	$\mathbf{5.23 \pm 0.14}$	12.70 ± 0.11	12.04 ± 0.22
e_ℓ RMSE [cm]	$\mathbf{1.44 \pm 0.02}$	1.60 ± 0.04	1.57 ± 0.04
$\lvert\boldsymbol{v}\rvert_{\max}$ [m/s]	**11.19**	11.00	11.09
$\lvert\boldsymbol{a}\rvert_{p99}$ [m/s^2]	29.7 ± 0.4	29.9 ± 0.4	29.7 ± 1.0
$\lvert\boldsymbol{\omega}\rvert_{\text{rms}}$ [rad/s]	5.28 ± 0.03	5.68 ± 0.02	5.73 ± 0.11
L_p [m/lap]	22.44 ± 0.06	$\mathbf{21.80 \pm 0.06}$	21.80 ± 0.07
solve median [ms]	**5.88**	7.60	+0.2
solve p_{99} [ms]	**7.64**	9.63	+0.2
solid-gate contacts (n=100)	17/100	**2/100**	0/100

The optimized gain vector $\boldsymbol{\theta} = \left[q_\phi,\, q_c,\, q_\ell,\, w_\omega,\, w_s,\, v_{s,\max}\right]$ maps to the cost weights of (18) as $\boldsymbol{Q}_\phi = q_\phi \boldsymbol{I}_3$, $\boldsymbol{Q}_c = q_c \boldsymbol{I}_3$, $\boldsymbol{Q}_\ell = q_\ell \boldsymbol{I}_3$, $\boldsymbol{W}_\omega = w_\omega \boldsymbol{I}_3$, and $W_s = w_s$. $W_{\Delta f}$ is fixed at 10^{-3}, not searched.

Racing objectives. Gate crossings are sign changes of $\boldsymbol{n}_k^\top(\boldsymbol{p} - \boldsymbol{g}_k)$ within $2R_{\text{gate}}$ of the gate center; after entry, the next N_ℓ+1 crossings of G_0 define the racing laps. The tuning problem is formulated as the bi-objective optimization

$$\min_{\boldsymbol{\theta}} \left(T_{\min}(\boldsymbol{\theta}) + \mathcal{P}(\boldsymbol{\theta}),\ \delta_{\max}^{\text{worst}}(\boldsymbol{\theta})\right), \tag{26}$$

where $\mathcal{P}$ penalizes gate safety, completion, body rate, and jerk violations, applied only to the lap-time objective, keeping the worst-case gate offset unpenalized and the trade-off between speed and gate-crossing margin explicit.

Gain selection. For each controller, the ten fastest configurations whose worst tuning offset remains within R_{safe} are screened over five full-path solid-gate SiL runs on the onboard computer, and the fastest candidate completing all five is selected provisionally. Deployment further requires $\delta_{\max} \le R_{\text{safe}}$ in the n=100 validation of Section VI, which extends the n=30 screening and includes its first 30 runs. This rule rejects all seven non-dominated baseline configurations faster than the deployed gains. The complete final Pareto-front results, selected controller hyperparameters, configuration-selection procedure, and implementation are available in the public repository listed under Supplementary Material on the first page.

VI. Simulation Results

We evaluate the deployed gains in SiL over n=100 Monte Carlo seeds (pass-through gates; injected odometry noise, σ=1 cm, 0.5°, 0.01 m/s, 0.035 rad/s; randomized warm-start; Section V-A); Table III reports medians over completed runs: of 100 attempts, 8 and 10 are lost to a faulty simulator reset and 92 and 90 complete the laps; the campaign with solid gates below was run until 100 started correctly.

Lap time. DQ-MPCC reduces $T_{\min}$ by 6.7% (3.46 → 3.23 s) along a 2.9% shorter racing path (L_p=21.8 vs. 22.4 m/lap). The reduction comes with a similar $\lvert\boldsymbol{a}\rvert_{p99}$, within the spread (29.7 vs. 29.9 m/s^2), and a slightly higher body rate.

Scope of the comparison. Both prediction models describe the same rigid-body physics (Section V), but a single real-time iteration linearizes coordinate-dependent expressions: the attitude enters the translational dynamics with the thrust prefactor f/m (thrust-to-weight ratio up to 3 in the racing laps) in (4) and with the constant $\|\boldsymbol{g}\|$ in (8). The controllers were tuned independently (contouring-to-lag weight ratio: 287 versus 1.3), so the comparison evaluates their selected operating points rather than an isolated representation effect. On the tuning fronts the baseline reaches lower lap times at every matched level, but every such configuration fails the deployment rule, and exchanging the gain sets leaves neither controller deployable.

Runtime and constraints. The dual-quaternion state adds $\sim$ 1.7 ms of solve time; both controllers run without solver failures (one baseline cycle excepted). Over the racing laps, 10 ms solve overruns are rare (0.01% MPCC, 0.45% **DQ-MPCC**; the previous command persists); neither saturates its authority in simulation (p_{99}: thrust $\le$64% of $f_{\max}$, body rate $\le$76% of $\omega_{\max}$). Over the racing laps the tube is never violated and the lag bound is exceeded in 4.1% and 0.003% of samples (worst 53 and 4 cm), the baseline therefore running with its lag bound relaxed over part of the laps; with transients the tube is violated in 0.59% and 0.39% (worst 96 and 24 cm). The predicted states are not projected: in ten replayed SiL runs, over the racing laps they stay within 1.3×10^{-2} of unit norm and 2.4×10^{-2} of dual orthogonality, with the first node rebuilt every cycle. Solve times are measured on the deployment computer against the simulated plant, at the clock and solver build of the flights.

Contour precision. In simulation the baseline tracks the path more tightly, in contour RMSE (5.2 vs. 12.7 cm) and crossing offset ($\delta_{\max}$=17.9 vs. 24.5 cm): **DQ-MPCC** is faster but not uniformly more accurate, every crossing still inside R_{safe}.

Exact $\boldsymbol{J}_l^{-1}$ ablation. The approximation of Section IV-D is not negligible here: attitude errors to the nearest path point average ≈16° (p_{99}: 52–56°) in simulation and flight, for which $\|\boldsymbol{J}_l^{-1}(\boldsymbol{\phi}^e)\boldsymbol{t}^e - \boldsymbol{t}^e\|$ averages 11% of $\|\boldsymbol{t}^e\|$ (p_{99}: 42–46%). At the deployed gains the exact $\boldsymbol{J}_l^{-1}$ gives a lower median lap time, similar tracking metrics, zero rather than 2/100 solid-gate contacts, adds approximately 0.2 ms of solve time, and leaves 5/100 pass-through runs unfinished. This fixed-gain sensitivity test uses gains tuned for the deployed approximation, not an independently retuned comparison; the main comparison uses the slower deployed variant.

Solid-gate evaluation. For this metric, a solid-gate contact is counted when the vehicle reaches a solid gate ring and the simulated attempt ends in a fall. The counts cover the complete run: all 17 baseline events occurred during post-lap deceleration and both **DQ-MPCC** events during entry, outside the timed racing laps. These events are distinct from the failed real-world racing attempts.

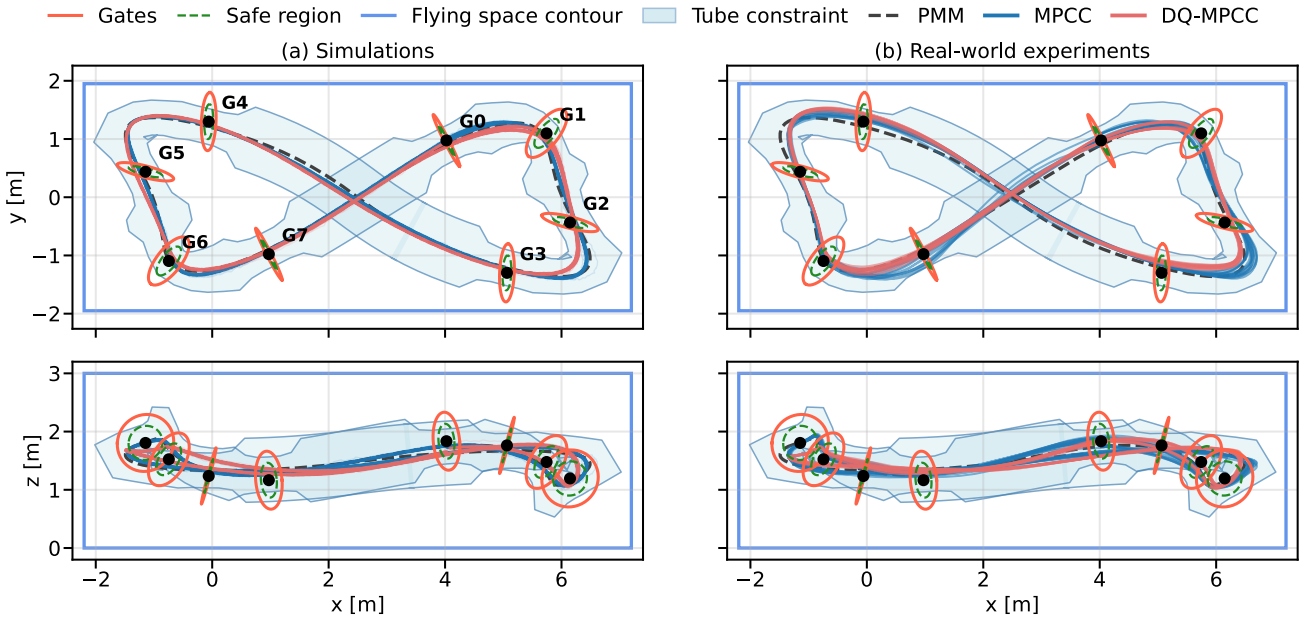


Fig. 4: Racing laps for MPCC vs **DQ-MPCC** over the tube and gate rings, XY (top) and XZ (bottom): (a) software-in-the-loop, 92 and 90 completed runs of the n=100 campaigns; (b) Real-world experiments, six flights per controller.

TABLE IV: Real-world racing attempts: completed three-lap flights and failed attempts following reported solver failures.

	attempts	compl. (3 laps)	failed att.	margin comp.
MPCC	11	6	5	2/6
DQ-MPCC	7	6	**1**	**6/6**

VII. Real-World Experiments

The platform is a custom quadrotor with an onboard NVIDIA Jetson Orin NX running the control optimization at 100 Hz, a Betaflight autopilot with incremental nonlinear dynamic inversion (INDI) body-rate control, and external motion-capture state at 240 Hz. Both controllers fly the circuit of Fig. 1 with the SiL gains and no retuning. SiL lap-time, tracking, and offset metrics (Figs. 5, 7) use non-colliding rings, since offsets beyond R_{safe} are observable only without contact; solid-gate contact rates come from separate contact campaigns. Table IV reports the trial accounting: in-air racing attempts continued until six flights per controller completed the three analysis laps (18 laps per controller; Fig. 4) from a fixed warm-start hover. MPCC had five failed racing attempts at racing speed and **DQ-MPCC** had one below racing altitude, each after a solver failure reported by the controller. These failed real-world racing attempts are separate from the SiL solid-gate contacts.

The real world preserves the lap-time ranking (-10.5%, larger than SiL; Fig. 5) but the worst-case gate offset *reverses*: **DQ-MPCC** keeps every gate inside R_{safe} (6/6 flights, $\delta_{\max}$=22.1 cm), whereas the MPCC baseline, tighter in simulation, now deviates by 30.7 cm, failing crossing-margin compliance on 4 of 6 flights (Fig. 6). The reversal is confined to the worst gate: the baseline keeps lower *average* offset and contour error (Table V).

The sim-to-real change in gate offset is accompanied by a different change in the actuation envelope: under real-world

TABLE V: RW racing metrics, median $\pm$ std over the six completed flights per controller. Δ vs MPCC; **bold** = better.

Metric	MPCC	**DQ-MPCC**	Δ
$T_{\min}$ [s]	3.872 ± 0.032	$\mathbf{3.464 \pm 0.010}$	$\mathbf{-10.5\%}$
T_{avg} [s]	3.906 ± 0.030	$\mathbf{3.493 \pm 0.009}$	$\mathbf{-10.6\%}$
$\delta_{\max}$ [cm]	30.7 ± 1.8	$\mathbf{22.1 \pm 1.0}$	$\mathbf{-28.1\%}$
δ_{avg} [cm]	$\mathbf{13.0 \pm 0.8}$	14.7 ± 0.4	$+13.1\%$
e_c RMSE [cm]	$\mathbf{11.5 \pm 0.9}$	14.0 ± 0.4	$+21.3\%$
e_ℓ RMSE [cm]	$\mathbf{4.59 \pm 0.01}$	4.82 ± 0.01	$+5.1\%$
$\lvert\boldsymbol{v}\rvert_{\max}$ [m/s]	$\mathbf{10.45 \pm 0.10}$	10.40 ± 0.06	-0.5%
$\lvert\boldsymbol{a}\rvert_{p99}$ [m/s^2]	34.9 ± 2.5	28.7 ± 0.2	-18.0%
$\lvert\boldsymbol{\omega}\rvert_{\text{rms}}$ [rad/s]	6.66 ± 0.26	6.47 ± 0.08	-2.9%
L_p [m/lap]	21.28 ± 0.08	$\mathbf{20.75 \pm 0.05}$	$\mathbf{-2.5\%}$

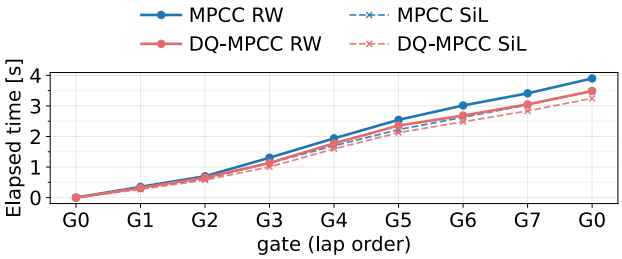


Fig. 5: Per-gate lap-time build-up (median over racing laps). **DQ-MPCC**'s lead over the baseline grows over the second half of the lap and is larger in the real world than in simulation.

mismatch and attitude lag, the baseline's $|\boldsymbol{a}|_{p99}$ increases by 17.5% sim-to-real (to 34.9 m/s^2), whereas **DQ-MPCC**'s decreases by 4.1% (28.7 m/s^2). In the real world the baseline's larger corrections coincide with crossings outside R_{safe}, latency and unmodeled dynamics acting jointly. The body-frame formulation carries a countervailing sensitivity: an attitude estimate error $\delta\boldsymbol{\theta}$ perturbs $\boldsymbol{v}_b = \boldsymbol{R}^\top \boldsymbol{v}$ by at most $\|\boldsymbol{v}\|\,\|\delta\boldsymbol{\theta}\|$, a sensitivity the inertial-coordinate baseline does not carry.

Table VI and Fig. 7 show the transfer without retuning: **DQ-**

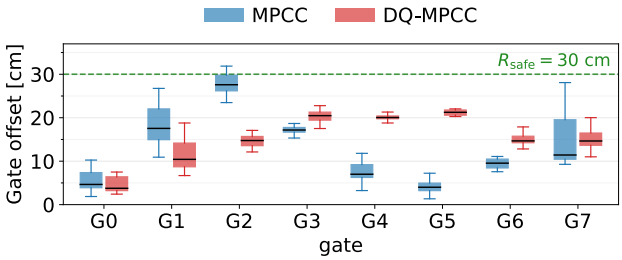


Fig. 6: Gate-crossing offset at each gate over the RW racing laps. **DQ-MPCC** stays inside R_{safe} at every gate; the baseline spreads to the boundary and crosses it at its worst gate, G_2.

TABLE VI: Sim-to-real gap (no retuning): per-controller RW-vs-SiL change. **bold**: smaller absolute RW-vs-SiL change. Relative changes use unrounded medians.

Ctrl.	$\Delta T_{\min}$	$\Delta\delta_{\max}$	Δe_c	$\Delta\lvert\boldsymbol{a}\rvert_{p99}$	$\Delta\omega_{\text{rms}}$	ΔL_p
MPCC	$+12.0\%$	$+71.5\%$	$+121\%$	$+17.5\%$	$+26.2\%$	-5.2%
DQ-MPCC	$\mathbf{+7.4\%}$	$\mathbf{-10.1\%}$	$\mathbf{+9.7\%}$	$\mathbf{-4.1\%}$	$\mathbf{+13.9\%}$	$\mathbf{-4.8\%}$

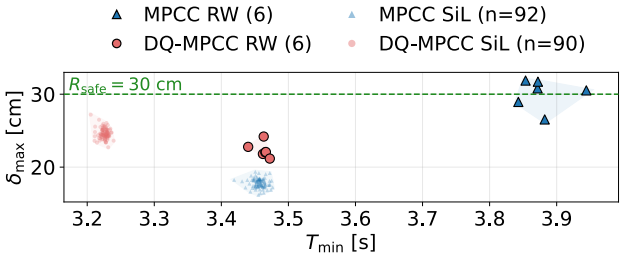


Fig. 7: Speed–crossing-margin plane: SiL clouds and RW flights (convex hulls). From simulation to the real world the baseline shifts across R_{safe} whereas **DQ-MPCC** stays inside it.

MPCC's sim-to-real changes in gate-crossing offset, contour error, and lap time remain small, whereas the baseline's worst-case gate offset increases by 71.5%, crossing R_{safe}.

VIII. Conclusion

We presented **DQ-MPCC**, combining dual-quaternion prediction, online progress optimization, and desired-body-frame lag–contouring errors. Running onboard at 100 Hz, it reduced the minimum lap time by 6.7% in simulation and 10.5% in real flights. Without retuning, six of seven **DQ-MPCC** attempts completed all laps with every crossing inside the prescribed margin; the baseline completed six of eleven attempts, with margin compliance in two of six completed flights. Because the controllers were tuned independently, this favorable operating point does not isolate an inherent representation advantage. The evaluation is limited to one motion-capture circuit, few flights, and no formal safety guarantee; future work targets onboard estimation, larger environments, and progress-consistent attitude references.

References

[1] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Sci. Robot.*, vol. 6, p. eabh1221, 2021.

[2] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, pp. 982–987, 2023.

[3] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Sci. Robot.*, vol. 8, p. eadg1462, 2023.

[4] D. Hanover *et al.*, "Autonomous drone racing: A survey," *IEEE Trans. Robot.*, vol. 40, pp. 3044–3067, 2024.

[5] R. Penicka and D. Scaramuzza, "Minimum-time quadrotor waypoint flight in cluttered environments," *IEEE Robot. Autom. Lett.*, vol. 7, pp. 5719–5726, 2022.

[6] D. Hanover, P. Foehn, S. Sun, E. Kaufmann, and D. Scaramuzza, "Performance, precision, and payloads: Adaptive nonlinear MPC for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 7, pp. 690–697, 2022.

[7] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optim. Control Appl. Methods*, vol. 36, pp. 628–647, 2015.

[8] D. Lam, C. Manzie, and M. Good, "Model predictive contouring control," in *Proc. IEEE Conf. Decis. Control*, 2010, pp. 6137–6142.

[9] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Trans. Robot.*, vol. 38, pp. 3340–3356, 2022.

[10] J. Ji, X. Zhou, C. Xu, and F. Gao, "CMPCC: Corridor-based model predictive contouring control for aggressive drone flight," in *Proc. Int. Symp. Experimental Robotics (ISER)*, 2021, pp. 37–46.

[11] M. Krinner, A. Romero, L. Bauersfeld, M. Zeilinger, A. Carron, and D. Scaramuzza, "MPCC++: Model predictive contouring control for time-optimal flight with safety constraints," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[12] J. M. Selig, *Geometric Fundamentals of Robotics*. Springer, 2005.

[13] B. V. Adorno, "Robot kinematic modeling and control based on dual quaternion algebra – Part I: Fundamentals," hal-01478225, 2017.

[14] X. Wang and C. Yu, "Unit dual quaternion-based feedback linearization tracking problem for attitude and position dynamics," *Syst. Control Lett.*, vol. 62, pp. 225–233, 2013.

[15] H. Abaunza, P. Castillo, A. Victorino, and R. Lozano, "Dual quaternion modeling and control of a quad-rotor aerial manipulator," *J. Intell. Robot. Syst.*, vol. 88, no. 2–4, pp. 267–283, 2017.

[16] U. V. Kalabić, R. Gupta, S. Di Cairano, A. M. Bloch, and I. V. Kolmanovsky, "MPC on manifolds with an application to the control of spacecraft attitude on SO(3)," *Automatica*, vol. 76, pp. 293–300, 2017.

[17] S. Teng, D. Chen, W. Clark, and M. Ghaffari, "An error-state model predictive control on connected matrix Lie groups for legged robot control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2022, pp. 8850–8857.

[18] L. F. Recalde, D. Agrawal, J. Arrizabalaga, and G. Li, "DQ-NMPC: Dual-quaternion NMPC for quadrotor flight," *IEEE Robot. Autom. Lett.*, vol. 10, pp. 12 644–12 651, 2025.

[19] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.

[20] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 2520–2525.

[21] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proc. IEEE Conf. Decis. Control (CDC)*, 2010, pp. 5420–5425.

[22] H. Homburger, J. Frey, S. Wirtensohn, M. Diehl, and J. Reuter, "Millisecond NMPC for swing-up and stabilization of the Furuta pendulum in real world," *IEEE Trans. Control Syst. Technol.*, vol. 34, no. 1, pp. 545–551, 2026.

[23] F. Bullo and R. M. Murray, "Proportional derivative (PD) control on the Euclidean group," in *Proc. Eur. Control Conf.*, 1995, pp. 1091–1097.

[24] F. Bullo and A. D. Lewis, *Geometric Control of Mechanical Systems*. New York: Springer, 2005.

[25] J. Baumgarte, "Stabilization of constraints and integrals of motion in dynamical systems," *Comput. Methods Appl. Mech. Eng.*, vol. 1, no. 1, pp. 1–16, 1972.

[26] R. Verschueren *et al.*, "acados—A modular open-source framework for fast embedded optimal control," *Math. Program. Comput.*, vol. 14, pp. 147–183, 2022.

[27] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear MPC: bridging the gap via the real-time iteration," *Int. J. Control*, vol. 93, pp. 62–80, 2020.

[28] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD*, 2019, pp. 2623–2631.

[29] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, 2014.